Wouldn't it make sense to combine this page with https://trac.ffmpeg.org/wiki/denoise ?

[MasterQuestionable:
Perhaps.
While that one is more academic focused.
And has more rigorous standard on content quality.]

Denoise Examples

A denoise process will destructively and irreversibly modify a source. Common use-cases for denoise processing are:

- make the content more subjectively pleasing to the ear or eye
- remove digital noise
- remove noise inherited from an analog digitization
- remove hard-to-compress data to improve compression efficiency, at the cost of video fidelity.

Contents

Video Denoise Filters
Testing and Comparing Denoise filters
Visually identifying planes containing noise
Estimating noise with the bitplanenoise filter
Specifying planes
Comparing filters in vertical split-screen
Generating artificial noise
Example command lines for various denoise filters
atadenoise
bm3d
chromanr
dctdnoiz
fftdnoiz
hqdn3d
nlmeans
owdenoise
vaguedenoiser
Benchmarking the relative performance of denoise filters
Codec-specific denoisers

Video Denoise Filters

FFmpeg supports a variety of video denoise filters, each of own advantages and disadvantages:

- "atadenoise": Adaptive Temporal-Averaging denoiser
- "bm3d": Block-Matching 3D denoiser
- "chromanr": Reduces chrominance noise
- "dctdnoiz": Discrete Cosine Transform 2D denoiser
- "fftdnoiz": Fast Fourier Transform 3D denoiser
- "hqdn3d": High-quality 3D denoiser
- "nlmeans": Non-Local Means denoiser
- "owdenoise": Overcomplete-wavelet denoiser
- "removegrain": Spatial denoiser for progressive video
- "tmedian": Reduces noise by picking median pixels from several successive input video frames
- "tmix": Reduces noise by mixing successive video frames
- "vaguedenoiser": Wavelet-based denoiser

(sorted alphabetically)

This document does not offer direct recommendations for denoise video algorithm selection, since the expected quality and performance is entirely subjective and all sources differ.

- Users are encouraged to start with default values selected by the developer.
- Users are encouraged to preview and compare filters.
- Users are encouraged to preview on the type of device, resolution and environment where the content will most likely be consumed. It is noted that many televisions include runtime-denoise functions. If a digital television is the primary target device, users are encouraged to preview with and without television runtime-denoise processing enabled.
  [Such TV tend to screw up, regardless.]
  The quality of runtime-denoise algorithms in consumer electronics vary significantly in quality of implementation, and may interfere with comparative testing.

Testing and Comparing Denoise filters

Visually identifying planes containing noise

Prior to applying a denoise filter, a user is encouraged to separate and preview the individual planes and identify whether the source of the noise is in any particular plane.

For example, when an analog source is digitized, there may be a disproportionate amount of noise in the chroma plane.

The following command will produce a 4x4 mosaic containing `{original, luma, Cb, Cr}` planes, similar to plane preview-scopes in tools such as Neat Video noise-preview. Frames are converted to yuv444p (or yuv444p10le for 10bpc) in order to remove chroma-subsampling of a yuv422p or yuv420p source, where chroma-planes are encoded at a lower resolution.

```
$ ffplay "${In}" -vf "format=pix_fmts='yuv444p',split=4[split1][split2]
[split3][split4]; \
  [split2]extractplanes=planes='y'[y]; \
  [split3]extractplanes=planes='u',eq=gamma_b=1.5[u]; \
  [split4]extractplanes=planes='v',eq=gamma_r=1.5[v]; \
  [split1][y][v][u]xstack=inputs=4:grid='2x2'[out]"
```

The above command can be extended to view each `{ combined, luma, Cb, Cr }` plane using FFplay's "*cycle video filters or show modes*", using the `w` key.

```
$ ffplay "${In}" -vf "format=pix_fmts='yuv444p',split=4[split1][split2]
[split3][split4]; \
  [split2]extractplanes=planes='y'[y]; \
  [split3]extractplanes=planes='u',eq=gamma_b=1.5[u]; \
  [split4]extractplanes=planes='v',eq=gamma_r=1.5[v]; \
```

```
  [split1][y][v][u]xstack=inputs=4:grid='2x2'[out]" \
  -vf "extractplanes=planes='y',format=pix_fmts='yuv444p'[out]" \
  -vf "extractplanes=planes='u',format=pix_fmts='yuv444p',eq=gamma_r=1.5[out]"
\
  -vf "extractplanes=planes='v',format=pix_fmts='yuv444p',eq=gamma_b=1.5[out]"
```

Estimating noise with the bitplanenoise filter

The bitplanenoise filter can be used to indicate the level of noise within each bitplane. The filter is not authoritative or absolute, it is an estimation subject to the developer's algorithm. The bitplanenoise filter is useful for helping understand the effect of a denoise filter.

Each bitplane represents a different level of intensity.

- 8bpc (bit-per-component) video contains 8 bitplanes
- 10bpc video contains 10 bitplanes

The bitplanenoise filter can also be combined with the extractplanes filter for providing further granularity into each individual plane.

The following command will produce a mosaic, visually highlighting an estimation of the level of noise in each of 8 bitplanes.

```
$ ffplay "${In}" -vf "split=outputs=8[split1][split2][split3][split4][split5]
[split6][split7][split8]; \
  [split1]bitplanenoise=bitplane=1:filter=true[split1]; \
  [split2]bitplanenoise=bitplane=2:filter=true[split2]; \
  [split3]bitplanenoise=bitplane=3:filter=true[split3]; \
  [split4]bitplanenoise=bitplane=4:filter=true[split4]; \
  [split5]bitplanenoise=bitplane=5:filter=true[split5]; \
  [split6]bitplanenoise=bitplane=6:filter=true[split6]; \
  [split7]bitplanenoise=bitplane=7:filter=true[split7]; \
  [split8]bitplanenoise=bitplane=8:filter=true[split8]; \
  [split8][split7][split6][split5][split4][split3][split2]
[split1]hstack=inputs=8[out]"
```

The bitplanenoise filter outputs results for each of the plane.bitplane to frame metadata.

The values can be printed to the console with the following command...

```
$ ffplay "${In}" -vf "bitplanenoise=bitplane=4,metadata=mode=print"
```

Which prints the result for bitplane 4 for each of planes {0, 1, 2} representing the {y, u, v} planes in a YUV pixel format.

```
...
[Parsed_metadata_1] lavfi.bitplanenoise.0.4=0.011723
[Parsed_metadata_1] lavfi.bitplanenoise.1.4=0.025428
```

```
[Parsed_metadata_1] lavfi.bitplanenoise.2.4=0.023403
...
```

Specifying planes

Some denoise filters allow the user to select which planes are processed. Planes are typically decimal notation, which may be confusing to a new user. This notation is not unique to `avuy` formats, it is also used for `abgr` formats.

```
Decimal  avuy  Plane
-------------------------------------------------
15       1111  all planes, including alpha
7        0111  all planes, excluding alpha
6        0110  chroma planes
1        0001  luma plane only
```

Comparing filters in vertical split-screen

The visual effect of a denoise filter is subjective. A user is encouraged to compare the visual effects of denoise filters.

The following command provides a vertical split-screen comparison between `null` (no-operation) filter on the left of the player window with the `hqdn3d` filter (with default options) on the right of the vertical center-line.
The same methodology can be used to compare the visual effects of 2 different denoise filters, or 2 different options of the same filter.

```
$ ffplay "${In}" -vf "split=2[split1][split2]; \
  [split1]null[split1]; \
  [split2]hqdn3d[split2]; \
  [split1]crop=out_w='(iw/2)':x=0[split1]; \
  [split2]crop=out_w='(iw/2)':x='(iw/2)'[split2]; \
  [split1][split2]hstack=inputs=2[out]"
```

Generating artificial noise

When testing a denoise filter or workflow, it is useful to be able to create artificial or synthetic noise with the noise filter.

The following command will create temporal noise (t) in the luma plane (c0).

```
$ ffplay -sws_flags '+accurate_rnd+full_chroma_int' \
  -f 'lavfi' "testsrc2=size='ntsc':rate='ntsc', \
  format=pix_fmts='yuv422p', \
```

```
  setdar=ratio='(4/3)', \
  noise=c0_strength=5:c0_flags='t'[out]"
```

It is possible to visually confirm that the noise mostly exists the luma plane with a combination of the bitplanenoise and extractplanes filter scopes.

```
-vf
"bitplanenoise=bitplane=1:filter=true,format=pix_fmts='yuv444p',split=4[split1]
[split2][split3][split4]; \
  [split2]extractplanes=planes='y'[y]; \
  [split3]extractplanes=planes='u',eq=gamma_b=1.5[u]; \
  [split4]extractplanes=planes='v',eq=gamma_r=1.5[v]; \
  [split1][y][v][u]xstack=inputs=4:grid='2x2'[out]"
```

Likewise, the noise filter can be used to create uniform, spatial noise (u), this time in the chroma planes {c1, c2}.

```
$ ffplay -sws_flags '+accurate_rnd+full_chroma_int' \
  -f 'lavfi' "testsrc2=size='ntsc':rate='ntsc', \
  format=pix_fmts='yuv422p', \
  setdar=ratio='(4/3)', \
  noise=c0_strength=0:c1_strength=5:c2_strength=5:all_flags='u'[out]"
```

Example command lines for various denoise filters

atadenoise

Process video frames using the Adaptive Temporal Averaging (atadenoise) Denoise filter.

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf atadenoise "${Out}"

$ ffplay "${In}" -vf "atadenoise" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v
"atadenoise=0a=0.02:0b=0.04:1a=0.02:1b=0.04:2a=0.02:2b=0.04:s=9:p=7:a='p':0s=32
\
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=atadenoise
```

Supported pixel formats:
gray, gray9le, gray10le, gray12le, gray14le, gray16le,
yuv410p, yuv411p, yuv420p, yuv422p, yuv440p, yuv444p, yuvj420p, yuvj422p, yuvj440p, yuvj444p, yuvj411p, yuv420p9le, yuv422p9le, yuv444p9le, yuv420p10le, yuv422p10le, yuv444p10le, yuv440p10le, yuv444p12le, yuv422p12le, yuv420p12le, yuv440p12le, yuv444p14le, yuv422p14le, yuv420p14le, yuv420p16le, yuv422p16le, yuv444p16le,
gbrp, gbrp9le, gbrp10le, gbrp12le, gbrp14le, gbrp16le,
yuva420p, yuva422p, yuva444p, yuva444p9le, yuva444p10le, yuva444p12le, yuva444p16le, yuva422p9le, yuva422p10le, yuva422p12le, yuva422p16le, yuva420p9le, yuva420p10le, yuva420p16le,
gbrap, gbrap10le, gbrap12le, gbrap16le

Online doc: https://ffmpeg.org/ffmpeg-filters.html#atadenoise

bm3d

Process video frames using the Block-Matching 3D (bm3d) denoise filter.

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf bm3d "${Out}"

$ ffplay "${In}" -vf "bm3d" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v
"bm3d=sigma=1.0:block=8:bstep=4:group=1:range=9:mstep=1:thmse=0:hdthr=2.70:esti
\
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=bm3d
```

Supported pixel formats:
gray, gray9le, gray10le, gray12le, gray14le, gray16le,
yuv410p, yuv411p, yuv420p, yuv422p, yuv440p, yuv444p, yuvj420p, yuvj422p, yuvj440p, yuvj444p, yuvj411p, yuv420p9le, yuv422p9le, yuv444p9le, yuv420p10le, yuv422p10le, yuv444p10le, yuv440p10le, yuv444p12le, yuv422p12le, yuv420p12le, yuv440p12le, yuv444p14le,

yuv422p14le, yuv420p14le, yuv420p16le, yuv422p16le, yuv444p16le,
gbrp, gbrp9le, gbrp10le, gbrp12le, gbrp14le, gbrp16le,
yuva420p, yuva422p, yuva444p, yuva444p9le, yuva444p10le, yuva444p12le, yuva444p16le, yuva422p9le, yuva422p10le, yuva422p12le, yuva422p16le, yuva420p9le, yuva420p10le, yuva420p16le,
gbrap, gbrap10le, gbrap12le, gbrap16le

Online doc: https://ffmpeg.org/ffmpeg-filters.html#bm3d

chromanr

Reduce noise in the chroma planes.

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf chromanr "${Out}"

$ ffplay "${In}" -vf "chromanr" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v
"chromanr=thres=30:sizew=5:sizeh=5:stepw=1:steph=1:threy=200:threu=200:threv=20
\
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=chromanr
```

Supported pixel formats:
yuv420p, yuv422p, yuv440p, yuv411p, yuv444p, yuva420p, yuva422p, yuva444p, yuvj444p, yuvj440p, yuvj422p, yuvj420p, yuvj411p, yuv420p9le, yuv422p9le, yuv444p9le, yuv420p10le, yuv422p10le, yuv440p10le, yuv444p10le, yuv444p12le, yuv422p12le, yuv440p12le, yuv420p12le, yuv444p14le, yuv422p14le, yuv420p14le, yuv420p16le, yuv422p16le, yuv444p16le, yuva420p9le, yuva422p9le, yuva444p9le, yuva420p10le, yuva422p10le, yuva444p10le, yuva422p12le, yuva444p12le, yuva420p16le, yuva422p16le, yuva444p16le

Online doc: https://ffmpeg.org/ffmpeg-filters.html#chromanr

dctdnoiz

Process video frames using the Discrete Cosine Transform (DCT) 2D denoise filter.

Note: This filter internally operates in RGB colorspace.
It does not directly support YUV pixel formats. And is limited to 8bpc.

It is noted in the documentation that this filter "is not designed for real time" use: mostly due to performance cause.
[^ Likely implementation issue: this should be similar (or simpler) [1]? comparing to "fftdnoiz".]
Might worth notice to who considering performance in selection of denoise algorithms.
[[1]?
Unsure if "fftdnoiz" uses DCT or DFT:
FFT may compute both.]

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf dctdnoiz "${Out}"

$ ffplay "${In}" -vf "dctdnoiz" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v "dctdnoiz=sigma=0:overlap=-1:n=3" \
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=dctdnoiz
```

Supported pixel formats:
bgr24, rgb24, gbrp

Online doc: https://ffmpeg.org/ffmpeg-filters.html#dctdnoiz

fftdnoiz

Process video frames using the 3D Fast Fourier Transform (FFT) denoise filter.
"3D" here just means it additionally considers the time domain: i.e. use of temporal frames. (in addition to "2D")

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf fftdnoiz "${Out}"

$ ffplay "${In}" -vf "fftdnoiz" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v
"fftdnoiz=sigma=1:amount=1:block=32:overlap=0.5:method='wiener':prev=0:next=0:p
\
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=fftdnoiz
```

Supported pixel formats:
gray, gray9le, gray10le, gray12le, gray14le, gray16le,
yuv410p, yuv411p, yuv420p, yuv422p, yuv440p, yuv444p, yuvj420p, yuvj422p, yuvj440p, yuvj444p, yuvj411p, yuv420p9le, yuv422p9le, yuv444p9le, yuv420p10le, yuv422p10le, yuv444p10le, yuv440p10le, yuv444p12le, yuv422p12le, yuv420p12le, yuv440p12le, yuv444p14le, yuv422p14le, yuv420p14le, yuv420p16le, yuv422p16le, yuv444p16le,
gbrp, gbrp9le, gbrp10le, gbrp12le, gbrp14le, gbrp16le,
yuva420p, yuva422p, yuva444p, yuva444p9le, yuva444p10le, yuva444p12le, yuva444p16le, yuva422p9le, yuva422p10le, yuva422p12le, yuva422p16le, yuva420p9le, yuva420p10le, yuva420p16le,
gbrap, gbrap10le, gbrap12le, gbrap16le

Online doc: https://ffmpeg.org/ffmpeg-filters.html#fftdnoiz

hqdn3d

Process video frames using the High-quality 3D denoise filter.
Which might be useful for removing shimmer by applying temporal denoise: especially on sources with excessive temporal noise in chroma plane.

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf hqdn3d "${Out}"

$ ffplay "${In}" -vf "hqdn3d" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options:

```
$ ffmpeg -i "${In}" \
  -filter:v
"hqdn3d=luma_spatial=4.00:chroma_spatial=3.00:luma_tmp=6.00:chroma_tmp=4.5" \
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=hqdn3d
```

Supported pixel formats:
yuv420p, yuv422p, yuv444p, yuv410p, yuv411p, yuv440p, yuvj420p, yuvj422p, yuvj444p, yuvj440p, yuv420p9le, yuv422p9le, yuv444p9le, yuv420p10le, yuv422p10le, yuv444p10le, yuv440p10le, yuv444p12le, yuv422p12le, yuv420p12le, yuv440p12le, yuv444p14le, yuv422p14le, yuv420p14le, yuv420p16le, yuv422p16le, yuv444p16le

Online doc: https://ffmpeg.org/ffmpeg-filters.html#hqdn3d

Note:
At the time of writing (FFmpeg 7.0.1), the command-line interface help suggests that all options default to 0.
The FFmpeg online doc suggests that the default value of luma_spacial is `4.0` and all other default values are solved-for.
The contradiction in documentation is noted.

nlmeans

Process video frames using the Non-local means denoise filter.
Also available hardware-accelerated through `nlmeans_opencl`, `nlmeans_vulkan`.

Note:
While nlmeans is considered one of the most advanced denoise filters.
The current implementation may be ["is"] limited to 8bpc formats.
[^
The implementation "is" indeed 8bpc.
But what 10bpc etc. really matters may not really matter this case...
See also: https://trac.ffmpeg.org/ticket/11107#comment:1
(and some background on denoise in general)]

Dirk Farin writes of the nlmeans filter... "*It is also interesting to note that one can simply apply the filter on interlaced material. The algorithm itself is robust enough to handle this nicely without artifacts.*" [Ref]
It's suggesting that in Farin's opinion nlmeans can successfully denoise interlaced material without significantly affecting the interlaced structure of which.
.
For theoretical best quality, don't use such general purpose denoiser for deinterlacing.
Use dedicated deinterlacer first then apply denoise.

Farin is not suggesting that nlmeans be used as deinterlacer, rather that nlmeans is robust enough to be applied in fully-interlaced end-to-end workflow.
.
For this purpose, and best quality:

First deinterlace after processing then reinterlace: must it be interlaced. (doesn't make much sense though)

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf nlmeans "${Out}"

$ ffplay "${In}" -vf "nlmeans" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v "nlmeans=s=1.00:p=7:pc=0:r=15:rc=0" \
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=nlmeans
$ ffmpeg -hide_banner -h filter=nlmeans_opencl
$ ffmpeg -hide_banner -h filter=nlmeans_vulkan
```

Supported pixel formats:
yuv410p, yuv411p, yuv420p, yuv422p, yuv440p, yuv444p, yuvj444p, yuvj440p, yuvj422p, yuvj420p, yuvj411p, gray, gbrp

Online doc:
https://ffmpeg.org/ffmpeg-filters.html#nlmeans
https://ffmpeg.org/ffmpeg-filters.html#nlmeans_005fopencl
https://ffmpeg.org/ffmpeg-filters.html#nlmeans_005fvulkan

See also:

- https://dirk-farin.net/projects/nlmeans/
  Dirk Farin claims the authors (A. Buades, B. Coll, J.M. Morel) of "*Image data processing method by reducing image noise, and camera integrating means for implementing said method*", EP Patent 1,749,278 (Feb. 7), 2007 granted permission to release the filter under GPL.

owdenoise

Process video frames using the Overcomplete Wavelet denoiser.

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf owdenoise "${Out}"
```

```
$ ffplay "${In}" -vf "owdenoise" -vf "null"  # Press 【W】 to cycle filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v "fftdnoiz=owdenoise=depth=8:luma_strength=1:chroma_strength=1 " \
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=owdenoise
```

Supported pixel formats:
yuv444p, yuv422p, yuv420p, yuv411p, yuv410p, yuv440p, yuva444p, yuva422p, yuva420p, yuv420p9le, yuv422p9le, yuv444p9le, yuv420p10le, yuv422p10le, yuv444p10le, yuv440p10le, yuv444p12le, yuv422p12le, yuv420p12le, yuv440p12le, yuv444p14le, yuv422p14le, yuv420p14le, yuv420p16le, yuv422p16le, yuv444p16le

Online doc: https://ffmpeg.org/ffmpeg-filters.html#owdenoise

vaguedenoiser

Process video frames using the wavelet-based vaguedenoise denoise filter.

Simple usage with default options:

```
$ ffmpeg -i "${In}" -vf vaguedenoiser "${Out}"

$ ffplay "${In}" -vf "vaguedenoiser" -vf "null"  # Press 【W】 to cycle
filters.
```

Advanced options usage:

```
$ ffmpeg -i "${In}" \
  -filter:v
"vaguedenoiser=threshold=2:method='garrote':nsteps=6:percent=85:planes=7:type='
\
  -codec:v 'libx264' -pix_fmt 'yuv420p' \
  -codec:a 'copy' \
  "${Out}"
```

In-program help:

```
$ ffmpeg -hide_banner -h filter=vaguedenoiser
```

Supported pixel formats:
gray, gray9le, gray10le, gray12le, gray14le, gray16le,
yuv410p, yuv411p, yuv420p, yuv422p, yuv440p, yuv444p, yuvj420p, yuvj422p, yuvj440p, yuvj444p, yuvj411p, yuv420p9le, yuv422p9le, yuv444p9le, yuv420p10le, yuv422p10le, yuv444p10le, yuv440p10le, yuv444p12le, yuv422p12le, yuv420p12le, yuv440p12le, yuv444p14le, yuv422p14le, yuv420p14le, yuv420p16le, yuv422p16le, yuv444p16le,
gbrp, gbrp9le, gbrp10le, gbrp12le, gbrp14le, gbrp16le,
yuva420p, yuva422p, yuva444p, yuva444p9le, yuva444p10le, yuva444p12le, yuva444p16le, yuva422p9le, yuva422p10le, yuva422p12le, yuva422p16le, yuva420p9le, yuva420p10le, yuva420p16le,
gbrap, gbrap10le, gbrap12le, gbrap16le

Online doc: https://ffmpeg.org/ffmpeg-filters.html#vaguedenoiser

Benchmarking the relative performance of denoise filters

The performance of denoise filters varies depending on a number of factors, including algorithm, patch size, number of planes being processed, hardware etc. A user can measure and compare the relative performance of a denoise filter and the selected options using the "latency" benchmarking filter.

> For example, here is a comparison of the relative performance of the `hqdn3d` and the `nlmeans` filters across 1000 video frames. These comparisons and values are not recommendations, but are to demonstrate how algorithm and options selection can have a major impact on performance.

```
$ ffmpeg -an -dn -sn -i "${In}" -filter:v hqdn3d,latency,trim=end_frame=1000 -
f null -
 [Parsed_latency] Min latency: 156 bitrate=N/A speed=14.1x

$ ffmpeg -an -dn -sn -i "${In}" -filter:v nlmeans,latency,trim=end_frame=1000
-f null -
 [Parsed_latency] Min latency: 136 bitrate=N/A speed=0.087x

$ ffmpeg -an -dn -sn -i "${In}" -filter:v
nlmeans=s=1.00:p=5:pc=5:r=7:rc=7,latency,trim=end_frame=1000 -f null -
 [Parsed_latency] Min latency: 136 bitrate=N/A speed=0.387x

$ ffmpeg -an -dn -sn -i "${In}" -filter:v
nlmeans=s=1.00:p=15:pc=15:r=31:rc=31,latency,trim=end_frame=1000 -f null -
 [Parsed_latency] Min latency: 133 bitrate=N/A speed=0.0179x
```

This relative performance gauge can be used to benchmark the impact of filter (and options) selection.

Codec-specific denoisers

It is noted that some encoders include denoise functions as private codec options. Such denoisers tend to be typical ones simulatable with FFmpeg's.

For example, the external x264 encoder supports the `--nr` option, where a user may specify a value in the range of {0...65536}, with the default being 0 (disabled).

```
$ x264 --nr 200 -o "${Out}" "${In}"

$ ffmpeg -i "${In}" -codec:v 'libx264' -x264opts 'nr=200' "${Out}"
```

FFmpeg's range of denoise filters have the advantage of being codec-independent: can be evaluated independent of a transcode and offer a wider selection of algorithms.

Last modified on Oct 3, 2024, 12:17:25 AM

t a g s

denoise